Handoff 41: Learner tests against reference programs (master run WD5, ENG-10)

Date: 2026-11-02 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#46), not merged, not deployed. No migration, no new dependency, and no lesson uses it yet.

> **How to use this file:** upload it with `docs/plan/browser-checks.md` and handoff 29. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing a new kind of browser check for Python Academy: a learner writes
tests, and the check runs them against a correct program and against broken ones.
Look for tests that catch nothing but pass, correct tests that fail, and any
expected result of a hidden case reaching the browser.
```

Run plan details

Field	Value
Stage	WD5, ENG-10. Stopped earlier because its spec contradicted itself, then unblocked by the owner's decision 28
Base	`main`, at `1947f7a`. **Text conflicts with #40** (section 8)
Part 2 boxes relied on	**None**

1. Step 0

`git fetch --all`, `gh pr list --state all`: #40, #43, #44 and #45 open. This PR is the fifth open, which is the cap, so the run stops after it. Worktree `../python-academy-reftests`; the main folder was not switched.

2. Handoff record (build.md template)

Field	Value
Task	A check can run a learner's tests against reference programs, which they should pass or fail on
Human goal	Unit 8 can check that a learner's tests are good, by whether they catch bugs
Non-goals	Any lesson; ENG-08b; the page's own wording for tests beyond the output panel
Data and compatibility	No migration. New optional fields, sent only when set, so existing plans are unchanged
Deployment status	Not requested

Files changed

File	Why
`frontend/public/python-worker.js`, `check-runner.js`	Load a case's reference first, in the same namespace
`frontend/app/adventure.tsx`	The output panel says "your tests all passed" or "a test stopped here" for a visible case
`backend/app/content.py`, `browser_checks.py`	`reference` and `test_should`, their validation, the plan and the judge
`backend/tests/case_runner.py`	Loads the reference, as the worker does
`backend/tests/test_reference_checks.py` (new, 23 tests), `frontend/tests/reference-checks.spec.ts` (new, 8 tests), a fixture lesson and a recording	Section 6
`docs/plan/browser-checks.md`	The rules
`docs/run-log.md`, this handoff and its PDF	The record

3. Security note

Every reference reaches the browser, as decision 28 allows, so a learner can read them. **A correct reference must not be the lesson's answer.** Content validation can't check that, so authors must. **A hidden case's `test_should` is its expected result, and is not sent.** References run in the worker with the same limits and the same import guard as learner code (tested). Nothing runs on the server.

4. Decisions and why

Decision	Options considered	Reason	Who decided
Reference and tests share one fresh namespace	Import it as a module	Simple for a beginner, and needs nothing from ENG-08b	Implementer (**assumption 1**)
On a broken reference, any stop counts as caught	Only `AssertionError`	A broken program may crash under a good test. The first case proves the tests run on correct code, so a typo is reported there	Implementer (**assumption 2**)
The first case must be visible and correct	Any order	Otherwise a typo could look like a bug caught	Implementer
Output is ignored	Compare it too	The tests are what is being judged	Implementer
The reference's file name is `<reference>`	Share `<exec>`	A line number shown to a learner is always one of theirs	Implementer

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
Break B5 got past my tests: a refused import counted as a bug caught, and nothing noticed	Deliberate-break run	A new test, for a refused import on a broken reference alone. B5 now fails 1 test
One Playwright test failed once, cause not established: `unit3.spec.ts:90`, "Step by step: every card shows", in a lesson this PR doesn't change	Full suite, first run	Three later full runs passed. I only kept the last lines of that log. The machine's load average was 12 to 15, which may be why, **but that is unconfirmed.** Watch for it in CI
My edit script assumed older text in `check-runner.js` and stopped partway	The script's own check	Re-applied against the current text

6. How it was tested

Command	Result
Backend on SQLite	**828 passed, 11 skipped**
Backend on PostgreSQL	**839 passed**
`npx tsc --noEmit && npm run build`	OK
`CI=1 npx playwright test`	**287 passed,** three times in a row (see section 5)

Covered: good tests pass; `assert True`, no tests, and print-only tests fail; a test wrong on correct code says so; a missed hidden bug is named by its description with its code not shown; a typo is a Python error

on the learner's own line; each case gets only its own reference; the import guard applies to learners and references; a timeout is never a pass; 8 kinds of bad content stop loading.

Deliberate break (each undone)	Tests that failed
B1. The worker does not load the reference	**5**
B2. The runner drops the reference	**5**
B3. Tests that catch nothing count as good	**5**
B4. A test failing on correct code counts as good	**2**
B5. A refused import counts as a bug caught	**0 at first,** then **1** with the new test
B6. The plan sends a hidden `test_should`	**2**
B7. A check need not start on correct code	**2**

Mocks: page tests use recorded real answers, for exactly matching requests. Python, the worker and the page are real. **Not run:** an axe scan, as the only change to the page is text in the existing output panel.

7. Known limitations

- **A learner could read a broken reference** and aim a test at that bug alone. Practice evidence, as `browser-checks.md` says.
- **Tests that crash on anything still count on a broken program,** as long as they pass on the correct one.
- **Unit 8 still needs B-R2,** and its lessons aren't written.

8. Merging with #40 and #45

A trial merge of all five open PRs shows text conflicts between #40 and this PR in 5 files, and between #40 and #45 in `content.py`, all where each adds a line. GitHub will show them. **Merge #40 first, tell me, and I'll update #45 and this PR.** I have not yet tested those combinations.

9. Live checks for you

None: no real lesson uses it.

10. Questions for you

1. **Assumption 2:** on a broken program, any stop counts as caught. *Recommended, not yet confirmed: keep.*

2. **Assumption 1:** a shared namespace, so a learner's tests call `score(...)` with no import.
   Recommended, not yet confirmed: keep for unit 8.

11. For the docs sync

Where	What
Backlog ENG-10	proposed → in_review (#46, handoff 41)
Stage table	"41. ENG-10 | Learner tests against references | in_review (#46, handoff 41)"
`browser-checks.md`	Number the new section

12. Next concrete action

Merge #40, then tell me, and I'll update #45 and this PR.

13. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-reftests status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-reftests/frontend/AGENTS.md   # generated by Next.js, not part of
the project
git worktree remove ../python-academy-reftests     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
git branch -d eng/reference-tests                  # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that).